Performing Unions, Joins and Windowing in Spark

In this train, we'll explore how to perform advanced data operations such as unions, joins, windowing, and other modifications in Apache Spark.

Learning Objectives

By the end of this train, you should be able to:

- Understand and use unions, joins, windowing, and other modifications in Spark.

SQL-like transformations in Spark

Using SparkSQL allows us to use SQL-like operations to manipulate and transform data.

Let's create some mock datasets to play around with. We will use these to practice joining, create window functions and concatenate tables.

```
# We create a city info table that describes some static features of cities in
South Africa.

city_info = spark.createDataFrame([
    (0, 'Johannesburg', '2001', '011', 1886, False),
    (1, 'Pretoria', '0001', '012', 1855, False),
    (2, 'Durban', '4001', '031', 1880, True),
    (3, 'Cape Town', '8000', '021', 1652, True),
    (4, 'Port Elizabeth', '6001', '041', 1820, True),
    (5, 'Bloemfontein', '9300', '051', 1846, False),
    (6, 'Kimberley', '8301', '053', 1873, False),
    (7, 'East London', '5200', '043', 1847, True),
], ['index', 'city', 'postal_code', 'area_code', 'established', 'coastal'])

# Let's create a city data table which contains data from cities from a recent
census
city_data = spark.createDataFrame([
    (0, 'Johannesburg', 10500000, 76000000, 1644.98),
    (1, 'Pretoria', 2921612, 75600000, 687.54),
    (2, 'Durban', 3442361, 83900000, 225.91),
    (3, 'Cape Town', 3740026, 78700000, 2461.0),
    (4, 'Port Elizabeth', 1152915, 45600000, 251.03),
    (5, 'Bloemfontein', 747431, 15600000, 236.17),
    (6, 'Kimberley', 225160, 460000, 164.3),
    (7, 'East London', 755200, 23400000, 168.86),
], ['index', 'city', 'population', 'gdp', 'area'])

city_info.createOrReplaceTempView('city_info')
city_data.createOrReplaceTempView('city_data')
```

```
# Selecting data from city info.
spark.sql("SELECT * FROM city_info").show()
```

```
+-----+--------------+-----------+---------+-----------+-------+
|index|          city|postal_code|area_code|established|coastal|
+-----+--------------+-----------+---------+-----------+-------+
|    0|  Johannesburg|       2001|      011|       1886|  false|
|    1|      Pretoria|       0001|      012|       1855|  false|
|    2|        Durban|       4001|      031|       1880|   true|
|    3|     Cape Town|       8000|      021|       1652|   true|
|    4|Port Elizabeth|       6001|      041|       1820|   true|
|    5|  Bloemfontein|       9300|      051|       1846|  false|
|    6|     Kimberley|       8301|      053|       1873|  false|
|    7|   East London|       5200|      043|       1847|   true|
+-----+--------------+-----------+---------+-----------+-------+
```

```
# Selecting data from city data.
spark.sql("SELECT * FROM city_data").show()
```

```
+-----+--------------+----------+--------+-------+
|index|          city|population|     gdp|   area|
+-----+--------------+----------+--------+-------+
|    0|  Johannesburg|  10500000|76000000|1644.98|
|    1|      Pretoria|   2921612|75600000| 687.54|
|    2|        Durban|   3442361|83900000| 225.91|
|    3|     Cape Town|   3740026|78700000| 2461.0|
|    4|Port Elizabeth|   1152915|45600000| 251.03|
|    5|  Bloemfontein|    747431|15600000| 236.17|
|    6|     Kimberley|    225160|  460000|  164.3|
|    7|   East London|    755200|23400000| 168.86|
+-----+--------------+----------+--------+-------+
```

Union

A union is the joining of two tables with the same schema.

Let's duplicate the city_info table and add an identifier:

```
# We are adding a column to the city_info table here, assigning it an identifier
value of 1 or 2.

city_info_1 = city_info.withColumn('origin', F.lit(1))
city_info_2 = city_info.withColumn('origin', F.lit(2))
```

Now let's create two views from these Spark dataframes.

```
city_info_1.createOrReplaceTempView('city_info_1')
city_info_2.createOrReplaceTempView('city_info_2')
```

Now let's perform a union to concatenate these views.

```
# Using the city_info views just created, we can perform a union.

spark.sql("SELECT * FROM city_info_1 UNION SELECT * FROM city_info_2").show(18,
False)
```

```
+-----+--------------+-----------+---------+-----------+-------+------+
|index|city          |postal_code|area_code|established|coastal|origin|
+-----+--------------+-----------+---------+-----------+-------+------+
|0    |Johannesburg  |2001       |011      |1886       |false  |1     |
|0    |Johannesburg  |2001       |011      |1886       |false  |2     |
|4    |Port Elizabeth|6001       |041      |1820       |true   |1     |
|2    |Durban        |4001       |031      |1880       |true   |1     |
|7    |East London   |5200       |043      |1847       |true   |2     |
|3    |Cape Town     |8000       |021      |1652       |true   |1     |
|4    |Port Elizabeth|6001       |041      |1820       |true   |2     |
|1    |Pretoria      |0001       |012      |1855       |false  |2     |
|7    |East London   |5200       |043      |1847       |true   |1     |
|2    |Durban        |4001       |031      |1880       |true   |2     |
|5    |Bloemfontein  |9300       |051      |1846       |false  |1     |
|1    |Pretoria      |0001       |012      |1855       |false  |1     |
|6    |Kimberley     |8301       |053      |1873       |false  |2     |
|5    |Bloemfontein  |9300       |051      |1846       |false  |2     |
|6    |Kimberley     |8301       |053      |1873       |false  |1     |
|3    |Cape Town     |8000       |021      |1652       |true   |2     |
+-----+--------------+-----------+---------+-----------+-------+------+
```

We can also perform the union using the PySpark API:

```
# Using the city_info dataFrames from earlier, we can perform a union.

city_info_1.union(city_info_2).show(18, False)
```

```
+-----+--------------+-----------+---------+-----------+-------+------+
|index|city          |postal_code|area_code|established|coastal|origin|
+-----+--------------+-----------+---------+-----------+-------+------+
|0    |Johannesburg  |2001       |011      |1886       |false  |1     |
|1    |Pretoria      |0001       |012      |1855       |false  |1     |
|2    |Durban        |4001       |031      |1880       |true   |1     |
|3    |Cape Town     |8000       |021      |1652       |true   |1     |
|4    |Port Elizabeth|6001       |041      |1820       |true   |1     |
|5    |Bloemfontein  |9300       |051      |1846       |false  |1     |
|6    |Kimberley     |8301       |053      |1873       |false  |1     |
|7    |East London   |5200       |043      |1847       |true   |1     |
|0    |Johannesburg  |2001       |011      |1886       |false  |2     |
|1    |Pretoria      |0001       |012      |1855       |false  |2     |
|2    |Durban        |4001       |031      |1880       |true   |2     |
|3    |Cape Town     |8000       |021      |1652       |true   |2     |
|4    |Port Elizabeth|6001       |041      |1820       |true   |2     |
|5    |Bloemfontein  |9300       |051      |1846       |false  |2     |
|6    |Kimberley     |8301       |053      |1873       |false  |2     |
|7    |East London   |5200       |043      |1847       |true   |2     |
+-----+--------------+-----------+---------+-----------+-------+------+
```

As you can see in both cases we have concatenated the data into a singular Spark dataframe.

Joins

It is a common operation to join two tables together. By default, Spark performs an inner join, with other options available, including `cross`, `outer`, `full`, `full_outer`, `left`, `left_outer`, `right`, `right_outer`, `left_semi`, and `left_anti`. Let's join the city info and data tables:

```
spark.sql("SELECT * FROM city_info ci JOIN city_data cd ON ci.city = cd.city
").show(5, False)
```

```
+-----+------------+-----------+---------+-----------+-------+-----+------------
+----------+--------+-------+
|index|city        |postal_code|area_code|established|coastal|index|city
|population|gdp     |area   |
+-----+------------+-----------+---------+-----------+-------+-----+------------
+----------+--------+-------+
|1    |Pretoria    |0001       |012      |1855       |false  |1    |Pretoria
|2921612   |75600000|687.54 |
|2    |Durban      |4001       |031      |1880       |true   |2    |Durban
|3442361   |83900000|225.91 |
|3    |Cape Town   |8000       |021      |1652       |true   |3    |Cape Town
|3740026   |78700000|2461.0 |
|0    |Johannesburg|2001       |011      |1886       |false  |0
|Johannesburg|10500000  |76000000|1644.98|
|6    |Kimberley   |8301       |053      |1873       |false  |6    |Kimberley
|225160    |460000  |164.3  |
+-----+------------+-----------+---------+-----------+-------+-----+------------
+----------+--------+-------+
only showing top 5 rows
```

Again, we can do the same in PySpark:

```
joined_cities = city_info.join(city_data, on='city')
joined_cities.show(5, False)
```

```
+------------+-----+-----------+---------+-----------+-------+-----+----------+-
-------+-------+
|city
|index|postal_code|area_code|established|coastal|index|population|gdp     |area
|
+------------+-----+-----------+---------+-----------+-------+-----+----------+-
-------+-------+
|Pretoria    |1    |0001       |012      |1855       |false  |1    |2921612
|75600000|687.54 |
|Durban      |2    |4001       |031      |1880       |true   |2    |3442361
|83900000|225.91 |
|Cape Town   |3    |8000       |021      |1652       |true   |3    |3740026
|78700000|2461.0 |
|Johannesburg|0    |2001       |011      |1886       |false  |0    |10500000
|76000000|1644.98|
|Kimberley   |6    |8301       |053      |1873       |false  |6    |225160
|460000  |164.3  |
+------------+-----+-----------+---------+-----------+-------+-----+----------+-
-------+-------+
only showing top 5 rows
```

It's just as easy as joining data in Pandas. Also, note that the column you specify to join on will only be represented once.

Windowing

A windowing function takes values from rows in a window (which is a limited number of rows) and then returns values, typically as a row. Note that, while taking in a set of rows, it is still possible to return a single value.

Windowing may require that we explain it in a bit more detail. Initially, Spark supported two types of transformations:

- Functions that take a single row as input and then generate a single value as an output.
- Functions that perform aggregations, take in a group of rows, and then calculate a single return value for each of the groups.

This left us with a group of operations not yet possible in Spark, operating on a group of rows but then returning a different value for each group of rows. Possible examples include moving averages, ranking groups, cumulative sums, and creating lags in time series.

The window function calculates a value for each row in a `Frame` (the group defined for the window).

We gain access to the `Window` function through the Spark SQL interface:

```
from pyspark.sql.window import Window
```

Let's illustrate its functionality with a couple of examples:

Ranking

First, let's use the rank function. Here we use `dense_rank()` to rank the cities by population, windowed by if they are coastal or not.

Let's first implement it in Spark SQL:

```
joined_cities.createOrReplaceTempView("joined_cities")
```

```
spark.sql("""SELECT
    *,
    dense_rank() OVER (PARTITION BY coastal ORDER BY population DESC) as
pop_rank_area
  FROM joined_cities
""").show()
```

```
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+-------------+
|
city|index|postal_code|area_code|established|coastal|index|population|     gdp|
area|pop_rank_area|
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+-------------+
|     Cape Town|    3|       8000|      021|       1652|   true|    3|
3740026|78700000| 2461.0|            1|
|        Durban|    2|       4001|      031|       1880|   true|    2|
3442361|83900000| 225.91|            2|
|Port Elizabeth|    4|       6001|      041|       1820|   true|    4|
1152915|45600000| 251.03|            3|
|   East London|    7|       5200|      043|       1847|   true|    7|
755200|23400000| 168.86|            4|
|  Johannesburg|    0|       2001|      011|       1886|  false|    0|
10500000|76000000|1644.98|            1|
|      Pretoria|    1|       0001|      012|       1855|  false|    1|
2921612|75600000| 687.54|            2|
|  Bloemfontein|    5|       9300|      051|       1846|  false|    5|
747431|15600000| 236.17|            3|
|     Kimberley|    6|       8301|      053|       1873|  false|    6|
225160|  460000|  164.3|            4|
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+-------------+
```

Spark SQL makes it quite intuitive. We are selecting everything using `SELECT *`, then we call the `dense_rank()` function, which requires that we specify over what we `PARTITION` the data and what we `ORDER` the data by. Now let's do the same in PySpark:

To create a `Frame` to partition by, we need to instantiate the `Window` class, # and use the partitionby() and orderby() methods to define the `Frame`.

```
coastal_population_window = Window.partitionBy(joined_cities['coastal']) \
                                  .orderBy(joined_cities['population'].desc())
```

We can then use the instantiated Window class to aggregate over.

```
joined_cities = joined_cities.withColumn('pop_rank_area',
F.dense_rank().over(coastal_population_window))
```

A lot is happening here. Let's look at it part by part. **First**, we call the `partitionBy()` method, which defined the field to partition by (here we select the `coastal` field). *Remember that you can also use F.col('coastal') to reference the fields.* **Next**, we define the field by which we want to order the rows within each `Frame`. **Finally**, we bring it all together by creating a new column using the `withColumn()` method. Here we call the `dense_rank()` function, and it has the `over()` method that takes the window object as an argument. This means that we will apply a dense rank over the specified window frame, effectively aggregating within the window frame. Overall, the result is quite the same between SparkSQL and PySpark, the interface is slightly different, and you can decide which works best for you. Going forward in this section, we are going to use PySpark implementations, which is more in line with the Python environment we have been using until now.

```
joined_cities.show()
```

```
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+-------------+
|
city|index|postal_code|area_code|established|coastal|index|population|     gdp|
area|pop_rank_area|
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+-------------+
|     Cape Town|    3|       8000|      021|       1652|   true|    3|
3740026|78700000| 2461.0|           1|
|        Durban|    2|       4001|      031|       1880|   true|    2|
3442361|83900000| 225.91|           2|
|Port Elizabeth|    4|       6001|      041|       1820|   true|    4|
1152915|45600000| 251.03|           3|
|   East London|    7|       5200|      043|       1847|   true|    7|
755200|23400000| 168.86|           4|
|  Johannesburg|    0|       2001|      011|       1886|  false|    0|
10500000|76000000|1644.98|           1|
|      Pretoria|    1|       0001|      012|       1855|  false|    1|
2921612|75600000| 687.54|           2|
|  Bloemfontein|    5|       9300|      051|       1846|  false|    5|
747431|15600000| 236.17|           3|
|     Kimberley|    6|       8301|      053|       1873|  false|    6|
225160|  460000|  164.3|           4|
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+-------------+
```

What happened here? We calculated the pop_rank_area, which is a rank based on if a town is coastal or not. You will see that coastal cities are ranked 1 to 4, and so also inland cities. This is all because we partitioned by 'coastal' and ordered by 'population'. So you will see that the rank is based on population.

Next, let's not just rank, but rather get into some analytics. Let's calculate the area difference between each city and the largest city for inland and coastal cities.

```
# Again using the Window class, partitioning by coastal and ordering by area.


coastal_area_window = Window.partitionBy(F.col('coastal')) \
  .orderBy(F.col('area').desc())

# This time, we are using arithmetic to calculate how much smaller the area is
# for the city in question than the largest one in the coastal or inland group.
joined_cities = \
  joined_cities.withColumn('coastal_rank_area',
F.max(F.col('area')).over(coastal_area_window) - F.col('area'))
```

```
joined_cities.show()
```

```
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+------------+-----------------+
|
city|index|postal_code|area_code|established|coastal|index|population|     gdp|
area|pop_rank_area|coastal_rank_area|
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+------------+-----------------+
|     Cape Town|    3|       8000|      021|       1652|   true|    3|
3740026|78700000| 2461.0|           1|              0.0|
|Port Elizabeth|    4|       6001|      041|       1820|   true|    4|
1152915|45600000| 251.03|           3|          2209.97|
|        Durban|    2|       4001|      031|       1880|   true|    2|
3442361|83900000| 225.91|           2|          2235.09|
|   East London|    7|       5200|      043|       1847|   true|    7|
755200|23400000| 168.86|           4|          2292.14|
|  Johannesburg|    0|       2001|      011|       1886|  false|    0|
10500000|76000000|1644.98|           1|              0.0|
|      Pretoria|    1|       0001|      012|       1855|  false|    1|
2921612|75600000| 687.54|           2|           957.44|
|  Bloemfontein|    5|       9300|      051|       1846|  false|    5|
747431|15600000| 236.17|           3|          1408.81|
|     Kimberley|    6|       8301|      053|       1873|  false|    6|
225160|  460000|  164.3|           4|          1480.68|
+--------------+-----+-----------+---------+-----------+-------+-----+----------
+--------+-------+------------+-----------------+
```

It is quite clear that Cape Town and Johannesburg are massive! Maybe coastal vs. inland is not the correct group comparison here.

▼Exercise: High vs. Low GDP ✖

As an exercise, create another grouping: high vs. low GDP (you can create an arbitrary threshold). And perform similar comparisons as above to see if the differences are smaller or more comparable.

Conclusion

Spark's ability to perform SQL-like transformations, including unions, joins, and window functions, provides a powerful and flexible toolkit for data manipulation and analysis, enabling users to efficiently process and derive insights from complex datasets.

▼Key Takeaways

- **Takeaway 1:** Spark's SQL interface and DataFrame API seamlessly integrate familiar SQL operations like UNION and JOIN, allowing for efficient combination and manipulation of structured data.
- **Takeaway 2:** Window functions in Spark, accessible through the Window class, enable sophisticated analytical operations such as ranking and calculating differences within defined groups or partitions, greatly extending the capabilities of data analysis.